Développement de Logiciels Sûrs

LOS

Chapitre 3

Détails du Chapitre 3

- Introduction
- Méthode Formelle B
- Exemple Complet
- Obligations de Preuve
- Raffinement & Implémentation

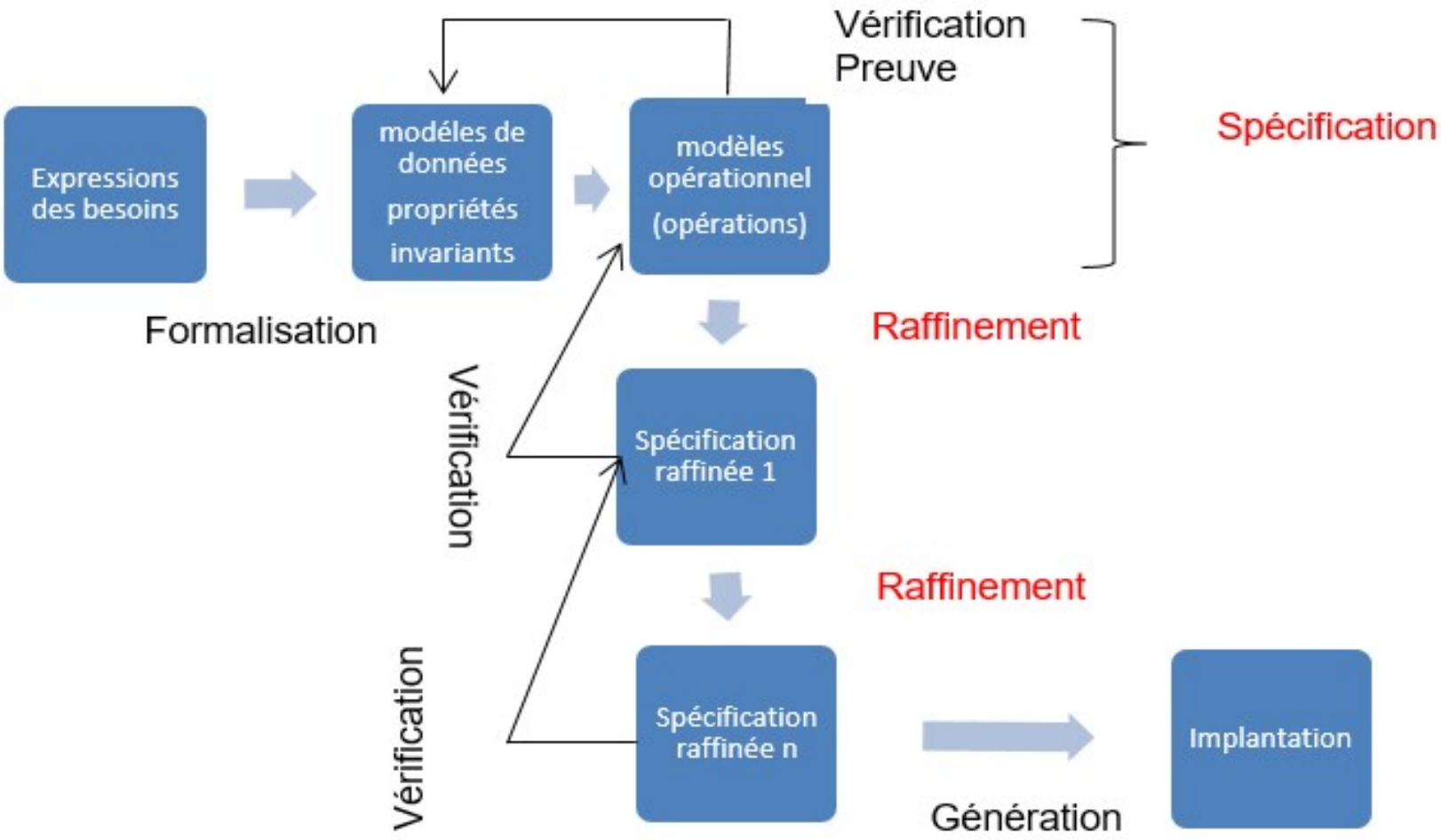
Introduction

- Le développement formel du logiciel permet le développement de programmes corrects par constructions. Une des méthodes de développement formel la plus utilisée, est le formalisme B.

- La méthode B :
  - C'est un langage de spécification (modélisation)
  - Une méthode formelle (fondations mathématiques) pour certifier :
    - La description des propriétés attendues
    - La description des fonctions (services) attendues
    - Un outil de vérification que les fonctions satisfont les propriétés
  - Une méthode de développement logiciel
    - Du cahier des charges au programme final
    - Une démarche progressive du modèle de haut niveau vers l'implantation
    - Une démarche préservant les propriétés du modèle sur l'implantation

Introduction

➔ Le processus de développement en B se résume comme suit :



Introduction

➔ Etat et espace d'états :

Considérons une variable dans un modèle logique

- Elle peut avoir plusieurs valeurs au cours du temps, ou plusieurs états.

  Par exemple : une variable entière i : on peut avoir i = 2, i = 6 … à condition que i soit modifié.

- Suite à une modification, on dit que i change d'état.

- Le changement d'état peut être modélisé par une action (substitue une nouvelle valeur à celle de la variable).

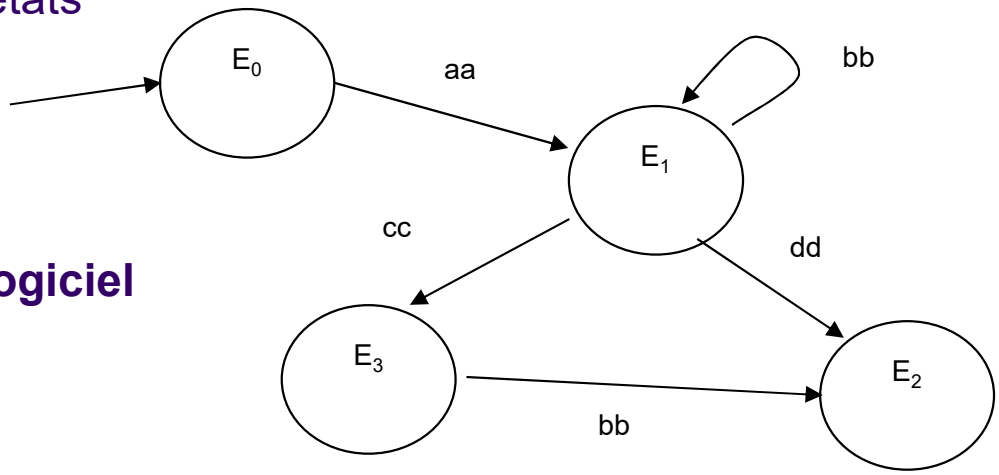
On parle d'espace d'états, l'ensemble des états autorisés pour la variable i.

Introduction

- **Exemple :**
  - *Un n-uplet de variables décrivant un état :*

    <mode = jour, lumière = éteint, temps = 20>
  - *Un prédicat (sur les variables) décrivant un espace d'état :*

    lumière = éteint ∧ mode = jour ∧ temps > 20
  - *L'opération mode := jour, affecte les variables ou change l'état du système.*
- L'approche de développement formel B est une approche orientée modèle à états :
  - Décrit un espace d'état
  - Décrit des opérations qui parcourent cet espace
  - Décrit un système (logiciel) de transitions entre états



Evolution d'un système logiciel

Méthode B : Fondements

Méthode B

- Brièvement :
  - Un langage et une méthode de spécification
  - Une méthode formelle : permettant de faire des preuves de validation si le logiciel répond correctement aux spécifications
  - Le langage est basé sur la théorie des ensembles (pour modéliser les données)
  - Offre une spécification simple décrite à l'aide d'une Machine Abstraite (MA)

Méthode B

- Une machine abstraite MA décrit :
  - L'état du système modélisé à travers des variables typées
  - Certaines de ses propriétés (**Invariant**)
  - Les services qu'il offre à travers les opérations

- Les constituants d'une machine abstraite sont :
  - Un état interne
  - Une initialisation
  - Des opérations
  - Des propriétés invariantes

Méthode B

→ Une machine abstraite MA décrit :

Modèle	Langage
Données (constantes + variables)	Ensemble
Initialisation / opérations	SG
Propriétés	Prédicat du 1er ordre

Méthode B

➔ Une machine abstraite MA :

```
MACHINE Nom (paramètres)

SETS S                          -- Ensembles

VARIABLES V                     -- Variables d'états, propriétés

INVARIANTS INV                  -- Propriétés d'invariance

INITIALISATION INIT

OPERATIONS op ← P|S

END;
```

Méthode B

- Une machine abstraite MA décrit :
  - Clause SETS
    - Permet de définir des ensembles abstraits ou énumérés. Ils sont utilisés pour typer les variables ; (ensembles prédéfinis notamment, NAT, INTEGER, …)
  - Clause INVARIANT
    - Permet de donner les propriétés invariantes de la machine : ce qui doit toujours être vrai et vérifié.

Méthode B

- Une machine abstraite MA décrit :
  - Interface des opérations
    - Aucun paramètres : NomOpération = …..
    - Que des paramètres d'entrée : NomOpération(p_1, p_2) = ….
    - Que des paramètres de sortie : r1, r2 ← NomOpération = ….
    - Paramètres d'entrée et sortie : r1, r2 ← NomOpération(p1,p2) = ….
  - Spécifier une opération = indiquer :
    - sa précondition
    - son body= effet sur les variables outputs et, éventuellement, mise à jour de l'état de la machine.

Méthode B

➔ **<u>Exemple 1 de Machine MA simple</u> :** Spécifiant le logiciel gérant une chaudière

MACHINE Chaudière

SETS

ETATS = {arm, dsm}

VARIABLES

T, Alarme

INVARIANT

$T \in NAT \ \wedge Alarme \ \in ETATS \ \wedge (T > 130) \Longrightarrow (Alarme = arm)$

OPERATIONS

ChangerT(v) =

PRE (precondition)

$v \in NAT \ \wedge v \leq 130$

THEN (body)

T := v

END

END

Méthode B

➔ **<u>Exemple 2 Machine Ticket</u> :** La machine distribue des billets (à la poste ou dans un supermarché) pour ordonner la queue des clients. A l'entrée, chaque client prend un ticket avec un nombre. Un écran montre le nombre du prochain client à servir.

```
MACHINE Tickets
VARIABLES
        serve, next                          -- serve : numéro du client à server (montré à l'écran)
                                             -- next : numéro du prochain ticket à donner
INVARIANT
        serve ∈ NAT ∧ next ∈ NAT ∧ serve ≤ next
INITIALISATION
                                 serve, next := 0,0
OPERATIONS
    ss ← serve_next =
        PRE
                        serve < next
                      THEN
         ss, serve := serve + 1, serve + 1
                      END
    tt ← take_next =
        PRE
         True                                -- PRE True peut-être omise
       THEN
         tt, next := next, next + 1
    END
END
```

Exemple Complet

Exemple Complet

- Il s'agit de décrire un système de réservation de places. Les places sont numérotées de 1 à nbmax. Sachant que la dernière valeur (nbmax) est un paramètre de la spécification.

- Les opérations offertes permettent :
  - De tester s'il reste des places libres : (opération place_libre)
  - De réserver une place (opération réserver)
  - De libérer une place déjà réservée (opération libérer)

Exemple Complet

Données :

- Paramètre nbmax vérifie la contrainte : $1 \leq nbmax \leq MAXINT$
- On définit l'ensemble SIEGE, comme l'intervalle 1…nbmax
- L'état du système est modélisé par un ensemble, <u>occupes</u>, contenant les places déjà allouées.
- On ajoute au modèle une variable <u>nblibre</u>, qui permet d'accéder au nombre de place libre.
- L'invariant s'écrit :

$$occupes \subseteq SIEGES$$
$$nblibre \in 0 \ldots nbmax$$
$$nblibre = nbmax - card(occupes)$$

Exemple Complet

- **Dynamique :**
  - Initialisation : occupes :=∅ , nblibre := nbmax
  - Interface des opérations :

    Opération *place_libre* : renvoie la valeur de la variable nb_libre

    Opération *réserver* a une pré-condition : elle ne peut être appliquée que s'il reste des places libres. C'est une opération non déterministe (la politique d'allocation des sièges n'est pas précisée).

    Opération *libérer* a une pré-condition : la place doit être occupée.

Exemple Complet

MACHINE Reservation(nbmax)
CONSTRAINTS
$nbmax \in 1 \cdots MAXINT$
DEFINITIONS
SIEGES == (1….nbmax)
VARIABLES
occupes, nblibre
INVARIANT
$occupes \subseteq SIEGES$
$\wedge\ nblibre \in 0 \ldots nbmax$
$\wedge\ nblibre = nbmax - card(occupes)$
INITIALISATION
occupes := ∅ || nblibre := nbmax
OPERATIONS
nb ← place_libre =
BEGIN
nb := nblibre
END
pp ← reserver =
PRE card(occupes) ≠ nbmax THEN
ANY place WHERE place ∈ SIEGES – occupes THEN
pp := place || occupes := occupes ∪ {place} || nblibre := nblibre - 1
END
END
liberer(place) =
PRE place ∈ occupes THEN
occupes := occupes – {place} || nblibre:=nblibre + 1
END
END

Obligations de Preuve (PO)

Obligations de Preuve (PO)

➔ Le développeur d'une machine MA a 2 types de PO (Proof Obligation) pour garantir la correction du modèle mathématique défini par la MA :

(1) Prouver que l'initialisation établit l'invariant
(2) Prouver que chaque opération, quand elle est appelée sous sa précondition, préserve l'invariant.

Obligations de Preuve (PO)

➔ Soit M une MA de la forme (la forme est simple pour alléger les PO) :

```
MACHINE M(p)
CONSTRAINTS C          -- (sur le parameter p)
VARIABLE x
INVARIANT I
INITIALISATION U
OPERATIONS
    r ← nom_op(w) = PRE P THEN S END ;
    …..
END
```

Dans la pratique, on s'équipe d'outils ou d'ateliers qui aident à décharger ces preuves

➔ **Les PO 1 et 2 s'écrivent :**

$$C \Longrightarrow [U]I$$ **(se lit « U établit I »)**

$$C \wedge I \wedge P \Longrightarrow [S]I$$

Obligations de Preuve (PO)

Preuve de l'exemple réservation :

L'invariant de la MACHINE est :

$occupes \subseteq SIEGES$ (4)

$\wedge\ nblibre \in 0 \ldots nbmax$ (5)

$\wedge\ nblibre = nbmax - card(occupes)$ (6)

Preuve de l'initialisation :

$nblibre$ est initialisée à nbmax

occupes est initialisé à $\emptyset$

Les PO s'écrivent :

$\emptyset \subseteq SIEGES$ (7)

$nb_max \in 0 \ldots nbmax$ (8)

$nb_max = nbmax - card(\emptyset)$ (9)

(8) nécessite de montrer que l'ensemble 0…nb_max est différent du vide. Ceci est vrai à partir de l'hypothèse nb_max appartient à 1… max(int)

Obligations de Preuve (PO)

Preuve de l'exemple réservation :

PO pour l'opération place_libre :

Cette opération ne modifiant aucune variable d'état de la machine, elle préserve donc l'invariant.

PO pour l'opération réservé :

Cette opération choisit une place non occupée. Puis décrémente le nombre de place libre.

Les PO s'écrivent :

$occupes \cup \{place\} \subseteq SIEGES$ (10)

$nblibre - 1 \in 0 \ldots nbmax$ (11)

$nblibre - 1 = nbmax - card(occupes \cup \{place\})$ (12)

On sait que :

$occupes \subseteq SIEGES$ (a)

$nblibre \in 0 \ldots nb_{max}$ (b)

$nblibre = nbmax - card(occupes)$ (c)

$card(occupes) \neq nb_max$ (d)

$place \in SIEGES - occcupes$ (e)

Ainsi

(a) + (e) $\Rightarrow$ (10)

(b) + nblibre $\neq$ 0 $\Rightarrow$ (11)

(c) + e $\Rightarrow$ (12)

Raffinement & Implémentation

Raffinement

- Le raffinement permet d'ajouter des précisions à une spécification, pour l'obtention du code exécutable. Il faut garantir que le raffinement est correct. En particulier il consiste à :
  - représenter concrètement les données abstraites (raffinement lié aux données)
  - choix de structures moins abstraites (raffinement lié aux données)
  - liaisons entre les variables concrètes et abstraites par un invariant de liaison (raffinement lié aux données)
  - réduire le non déterminisme (raffinement lié aux opérations)
  - affaiblissement des préconditions (raffinement lié aux opérations)

Raffinement

- Reprenons l'exemple de réservation. Le raffinement de cet exemple consiste à transformer :
  - L'ensemble abstrait (i.e., *occupes*) en une représentation concrète
  - L'opération « *réserver* » en une opération déterministe : choisir systématiquement la place ayant le plus petit numéro.

- Premier Point : l'ensemble *occupes* est représenté par une fonction totale de la forme :

$$\boldsymbol{état} \in \boldsymbol{SIEGES} \rightarrow \boldsymbol{BOOL}$$

A chaque siège est associé TRUE s'il est occupé, FALSE s'il est libre.

- La variable *occupes* peut être exprimée par :

$$occupes = etat^{-1}[\{TRUE\}]$$

- Cette formule correspond à l'invariant de liaison entre la machine et son raffinement.
- Aussi l'opération *libérer*, elle met systématiquement la valeur de l'état du siège libérer à FALSE.

Raffinement

```
REFINEMENT
        RESERVATION1(nb_max)
REFINES
        RESERVATION
DEFINITIONS
        SIEGES = (1 … nb_max)
VARIABLES
            Etat, nb_libre
INVARIANT
```

$$etat \in SIEGES \longrightarrow BOOL \wedge occupe = etat^{-1}[\{TRUE\}]$$

```
INITIALISATION
      nb_libre := nb_max    ||  etat :=SIEGES X {FALSE}
OPERATIONS
      /* l'opération nb_libre est héritée sans changement */
place ← réserver =
        BEGIN
            LET pp BE pp := min(etat^-1[{False}] )
      IN place := pp || nb_libre := nb_libre – 1 || état(pp) := TRUE
   END

libérer(place) =
   BEGIN
       etat(place) := FALSE || nb_libre := nb_libre + 1
   END
END
```

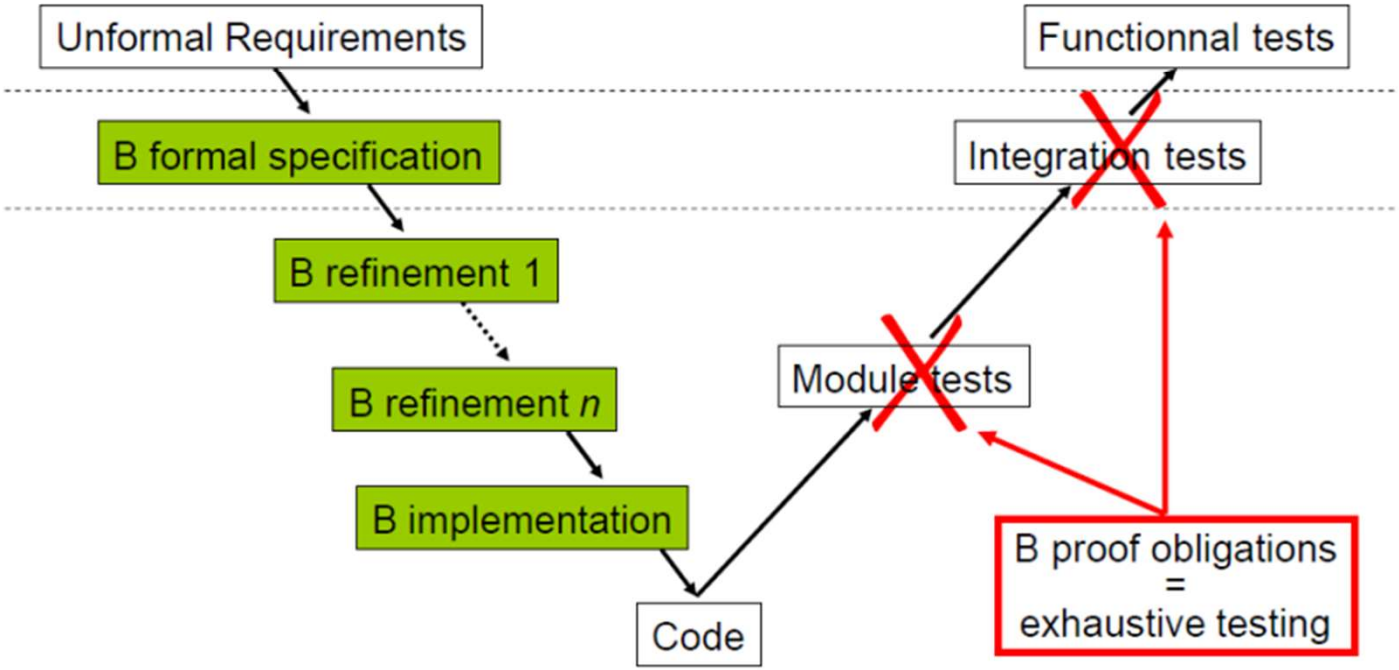
Implémentation

- Un programme est un raffinement particulier qui vérifie certaines contraintes correspondant à celles des langages de programmation classique. Il se présente avec le mot réservé **IMPLEMENTATION**. Par exemple, l'affectation simultanée est remplacée par un séquencement.

Méthode B en résumé ...

- Partir d'une spécification de *haut niveau*
  - Spécification *descriptive*
- *Ajouter des détails* jusqu'à arriver à un programme exécutable
  - Préciser l'*algorithme*
  - Préciser les *structures de données*

- *Avantages*
  - *Substituer* des preuves aux tests
    - *Exhaustivité* !
    - Coûts *réduits* !
  - Logiciel *correct par construction*
  - *Maintenance* facilitée



Succès de la Méthode B ...

➔ Ligne de métro 14 sans conducteur à Paris

- Environ 110 000 lignes de modèle B ont été écrites, générant environ 86 000 lignes de code Ada
  - ✓ **N.B.** : 10 à 50 erreurs pour 1000 lignes de code dans un logiciel classique

- Aucun bug trouvé après les preuves
  - ✓ Ni lors des tests d'intégration, des tests fonctionnels, des tests sur site ni depuis que la ligne est en opération (octobre 1998)
  - ✓ Le logiciel critique est toujours en version 1.0, sans bug détecté